Módulo 04

Clasificación y pruebas de hipótesis

Ciencia de Datos e IA Aplicada: de los datos a las decisiones

Semana 5 · 9 lecciones

Hoja de ruta del módulo

Parte 1 · Clasificación	Parte 2 · Hipótesis y A/B
4.1 Problemas de clasificación	4.7 H_o, p-valor, errores, potencia
4.2 Regresión logística	4.8 Pruebas A/B de principio a fin
4.3 Árboles de decisión	4.9 Trampas
4.4 Random forest y boosting	
4.5 Métricas, ROC y AUC	
4.6 El umbral como decisión	

Hilo conductor: **los errores tienen costes distintos**

4.1

Problemas de clasificación en la empresa

¿Qué es clasificar?

- Regresión (módulo 03): predecir un **número**
- Clasificación: predecir una **categoría**
- Binaria: clase **positiva (1)** y negativa (0)
- La positiva es la que **dispara una acción**, no «la mala»
- También: multiclase y multietiqueta

Clasificación en la empresa

Problema	Positivo (1)	Acción
Fraude	Transacción fraudulenta	Bloquear
Abandono	Cliente que se va	Oferta de retención
Crédito	Impago	Rechazar
Spam	Correo basura	A la papelera
Diagnóstico	Tumor maligno	Biopsia

De la probabilidad a la decisión

$$\hat{p} = P(Y = 1 \mid X) \qquad \hat{y} = \begin{cases} 1 & \text{si } \hat{p} \geq u \\ 0 & \text{si } \hat{p} < u \end{cases}$$

- Problema **estadístico**: estimar y ordenar bien → lecciones 4.2–4.4
- Problema **de decisión**: elegir el umbral u → lecciones 4.5–4.6
- `predict()` usa $u = 0{,}5$ por defecto

El desequilibrio de clases

- Fraude: 0,5 % · Abandono mensual: 2 % · Averías: < 1 %
- Modelo «nunca fraude» → **99,5 % de exactitud** y cero utilidad
- La exactitud deja de ser informativa
- Notebook: 569 tumores, **37 % malignos**
- Desequilibrio moderado, pero costes muy **asimétricos**

Cinco preguntas antes de modelar

1. ¿Qué es exactamente un positivo?
2. ¿Cuándo predigo y con qué información? (ojo a la **fuga**)
3. ¿Qué acción se dispara y con qué **capacidad**?
4. ¿Cuánto cuesta cada tipo de error?
5. ¿Cuál es la **línea base**?

Ejemplo: retención de clientes

Retener vale 300 € · la oferta cuesta 40 € · se marcan 1.000 clientes · la oferta retiene a la mitad

Bajas reales entre los 1.000	Retenidos	Resultado
200	100	30.000 − 40.000 = **−10.000 €**
400	200	60.000 − 40.000 = **+20.000 €**

La pregunta no es la exactitud: es **cuántos de los que marco son positivos**

4.2

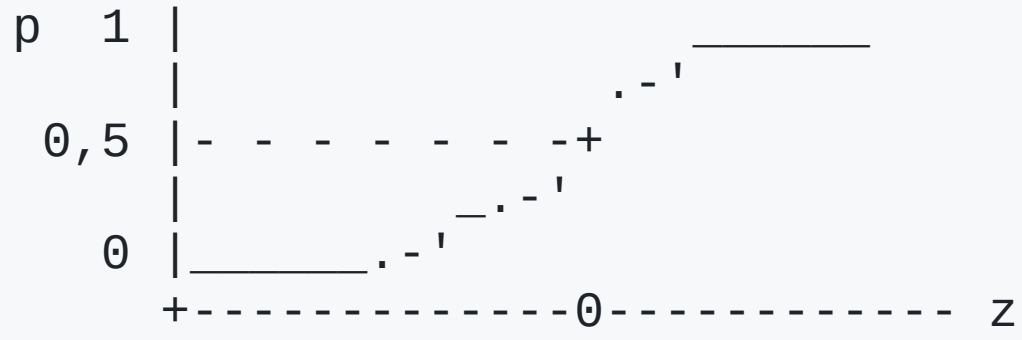
Regresión logística: probabilidades y odds

¿Por qué no una recta?

- Una regresión lineal sobre $Y \in \{0, 1\}$ da valores < 0 y > 1
- Eso no son probabilidades
- Idea: modelar linealmente **otra cosa**
- ...y transformarla en una probabilidad entre 0 y 1

La función sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}} \qquad \hat{p} = \sigma(\beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k)$$



Odds y log-odds

$$\text{odds} = \frac{p}{1-p} \qquad \log \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k$$

p	**odds**	**lectura**
0,20	0,25	«1 a 4»
0,50	1	a la par
0,80	4	«4 a 1»

Lo lineal es el **log-odds**

Interpretar coeficientes

- +1 unidad en x_j ⇒ log-odds $+\beta_j$ ⇒ odds $\times e^{\beta_j}$
- Ejemplo: reclamaciones, $\beta = 0{,}4$ ⇒ odds $\times 1{,}49$
- p: 0,10 → 0,14 · pero 0,50 → 0,60
- Efecto **constante en odds**, variable en probabilidad
- Correlación, no causalidad

Cómo se entrena

$$\mathcal{L} = -\frac{1}{n}\sum_{i}\left[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\right]$$

```
logit = make_pipeline(StandardScaler(),
                      LogisticRegression(max_iter=1000))
prob = logit.fit(X_train, y_train).predict_proba(X_test)[:, 1]
```

- Máxima verosimilitud = mínima *log loss*
- Regularización L2 por defecto (parámetro `C`)

Fortalezas y límites

✔ Fortalezas	✘ Límites
Interpretable (odds ratios)	Efectos lineales y aditivos
Probabilidades razonables	No capta interacciones sola
Rápida y estable	Coeficientes inestables con colinealidad
Notebook: **AUC 0,992** (el mejor)	

Línea base obligatoria en clasificación tabular

4.3

Árboles de decisión

La idea: preguntas sí/no

- Un diagrama de flujo aprendido de los datos
- Cada nodo: una pregunta sobre una variable
- Cada hoja: una predicción
- Probabilidad en la hoja = proporción de positivos de train
- El modelo más fácil de **explicar**

Cómo se construye (CART)

1. Prueba todas las variables y puntos de corte
2. Elige la división que deja hijos más **puros**
3. Repite en cada hijo hasta un criterio de parada

$$\text{Gini} = 2p(1-p)$$

Nodo puro: Gini 0 · mezcla 50/50: Gini 0,5

El árbol del notebook (profundidad 3)

```
mean concave points <= 0,05 ?
├─ sí → worst area <= 957 ?
│         ├─ sí → area error <= 45 ?         → benigno / maligno
│         └─ no → worst fractal dim <= 0,06 ? → benigno / maligno
└─ no → worst concave points <= 0,10 ?
          ├─ sí → benigno
          └─ no → worst area <= 710 ?        → benigno / maligno
```

Cada predicción se explica siguiendo un camino

Ventajas

- Interpretable si es pequeño
- **No hace falta escalar** variables
- Capta **no linealidades e interacciones**
- Se traduce en **reglas** de negocio
- Útil en sectores regulados (crédito, seguros, salud)

El problema: sobreajuste e inestabilidad

Hiperparámetro	Limita
`max_depth`	Preguntas encadenadas
`min_samples_leaf`	Casos mínimos por hoja
`min_samples_split`	Casos mínimos para dividir
`ccp_alpha`	Complejidad (poda)

Sin límites: memoriza el ruido · pocos datos distintos ⇒ árbol distinto

Resultado e importancia de variables

- Notebook: árbol prof. 3 → **AUC 0,916** (logística: 0,992)
- Solo 8 hojas ⇒ solo **8 probabilidades distintas** ⇒ ordena «a saltos»
- Importancia por impureza: útil para explorar, con sesgos
- Mejor: importancia por **permutación**
- Importancia ≠ causalidad

4.4

Ensembles: random forest y gradient boosting

La idea de los ensembles

- Un árbol es inestable → ¿y si entrenamos **muchos**?
- Combinar modelos con errores poco correlacionados
- Dos estrategias casi opuestas:
  - **Random forest**: en paralelo, se promedian
  - **Gradient boosting**: en secuencia, se corrigen

Random forest

1. **Bagging**: cada árbol, con una muestra bootstrap
2. En cada división, solo $\sqrt{k}$ variables al azar
3. Árboles profundos → se **promedian** las probabilidades

- Descorrelaciona los árboles → reduce **varianza**
- Más árboles nunca empeoran (solo tardan más)
- Error *out-of-bag* gratis

Gradient boosting

- Árboles **pequeños**, uno tras otro
- Cada árbol se ajusta a lo que les falta a los anteriores (gradiente de la pérdida)
- Parte de mucho **sesgo** y lo reduce paso a paso
- **Sí puede sobreajustar** → parada temprana
- Implementaciones: `HistGradientBoostingClassifier`, XGBoost, LightGBM, CatBoost

Boosting en una fórmula

$$F_M(x) = F_0(x) + \eta \sum_{m=1}^{M} h_m(x)$$

Hiperparámetro	Efecto
Nº de árboles M	Más → más ajuste (y riesgo)
Tasa de aprendizaje η	Menor → más prudente, más árboles
Profundidad / hojas	Complejidad de cada árbol

Random forest vs. gradient boosting

	Random forest	Gradient boosting
Construcción	Paralelo	Secuencial
Árboles	Profundos	Poco profundos
Reduce	Varianza	Sesgo
Ajuste	Poco sensible	Más sensible
Sobreajuste	Bajo	Moderado

Lo que dice el notebook (test, umbral 0,5)

Modelo	Exactitud	Recall	Precisión	AUC	FN
Logística	0,953	0,938	0,938	**0,992**	4
Árbol (prof. 3)	0,865	0,938	0,759	0,916	4
Random forest	0,942	0,938	0,909	0,980	4
Gradient boosting	**0,971**	**0,953**	**0,968**	0,987	**3**

Los ensembles **no siempre ganan** · con 171 casos, ±0,01 es ruido

¿Cuándo compensa la complejidad?

- Muchos datos, muchas variables, interacciones → ensembles
- Pocos datos, relaciones monótonas → logística compite
- Se pierde interpretabilidad directa → importancia, SHAP
- Ajusta hiperparámetros con **validación cruzada**, nunca con el test
- Elige también por mantenimiento y estabilidad

4.5

Matriz de confusión, precisión, recall, curva ROC y AUC

La matriz de confusión

	Predicho 0	**Predicho 1**
Real 0	TN (verdadero negativo)	FP (falso positivo)
Real 1	FN (falso negativo)	TP (verdadero positivo)

```
tn, fp, fn, tp = confusion_matrix(y, pred).ravel()
```

Depende del **umbral**

Logística en test (umbral 0,5)

	Pred. benigno	Pred. maligno
Benigno (107)	103	4
Maligno (64)	4	60

- 4 biopsias innecesarias
- **4 cánceres sin detectar**

Las métricas

Métrica	Fórmula	Valor
Exactitud	$(TP + TN)/N$	163/171 = 0,953
Precisión	$TP/(TP + FP)$	60/64 = 0,938
Recall	$TP/(TP + FN)$	60/64 = 0,938
Especificidad	$TN/(TN + FP)$	103/107 = 0,963
F1	media armónica prec./recall	0,938

¿Precisión o recall?

- **Precisión**: de los que marco, ¿cuántos son?
- **Recall**: de los que son, ¿cuántos marco?
- Marcar más ⇒ ↑ recall, ↓ precisión

Prioriza recall	Prioriza precisión
Cribado de cáncer	Filtro de spam
Fraude grave	Campañas con contacto caro
Seguridad	Alertas con equipo limitado

La curva ROC

- Un punto por **cada umbral**
- Eje X: tasa de falsos positivos $FP/(FP+TN)$
- Eje Y: recall $TP/(TP+FN)$
- Diagonal = azar · esquina (0,1) = perfecto

```
RocCurveDisplay.from_predictions(y_test, prob, name="Logística")
```

El AUC

> Probabilidad de que el modelo puntúe más alto a un **positivo** al azar que a un **negativo** al azar

- 0,5 = azar · 1 = perfecto
- Mide la **ordenación** → no depende del umbral
- No depende de la calibración
- Notebook: logística **0,992**

Positivos muy raros: curva precisión-recall

- 0,1 % de positivos · FPR del 1 % sobre 1 M de negativos
- = 10.000 falsas alarmas frente a 1.000 positivos
- Precisión ≤ 9 % aunque la ROC «parezca» buena
- Usa `precision_recall_curve` y *average precision*

Errores frecuentes

- Reportar solo exactitud con clases desequilibradas
- Confundir *precision* con *accuracy*
- «AUC = probabilidad de acertar» ✘
- Calcular métricas sobre el train
- Olvidar la incertidumbre: 1 FN más = −1,6 p.p. de recall con 64 positivos

4.6

El umbral es una decisión de negocio

0,5 no tiene nada de especial

Solo es razonable si:

- los dos errores **cuestan lo mismo**, y
- las probabilidades están **bien calibradas**

En la mayoría de problemas reales, no se cumple ninguna de las dos

La regla teórica

Marcar como positivo si $p\,C_{FN} > (1-p)\,C_{FP}$:

$$u^* = \frac{C_{FP}}{C_{FP} + C_{FN}}$$

$C_{FN} : C_{FP}$	u^*
1 : 1	0,50
5 : 1	0,17
20 : 1	0,048

La regla empírica (notebook §3)

```
prob_cv = cross_val_predict(logit, X_train, y_train, cv=cv,
                            method="predict_proba")[:, 1]
costes = [20 * ((prob_cv < u) & (y_train == 1)).sum()
          +  1 * ((prob_cv >= u) & (y_train == 0)).sum()
          for u in umbrales]
u_opt = umbrales[np.argmin(costes)]   # 0,06
```

- Predicciones de **validación cruzada** sobre el train
- **Nunca** elegir el umbral con el test

Umbral 0,5 vs. 0,06 (test)

Umbral	FN	FP	Recall	Coste (20:1)
0,50	4	4	0,938	84
0,06	**1**	18	**0,984**	**38**

63 de 64 cánceres detectados, a cambio de 14 biopsias más

¿Quién fija los costes? ¿Y otras formas?

- Los costes los fija **el negocio** (clínica, riesgos, finanzas)
- Tu trabajo: mostrar el compromiso con 3–4 umbrales
- Por **capacidad**: los 300 casos con mayor puntuación
- Por **restricción**: recall ≥ 95 %
- Por **beneficio esperado**: si los aciertos también valen
- `TunedThresholdClassifierCV` en scikit-learn

Ejemplo: microcréditos

- Impago no detectado (FN): **400 €**
- Buen cliente rechazado (FP): **60 €**

$$u^* = \frac{60}{60 + 400} \approx 0{,}13$$

La pregunta útil: ¿qué combinación de **impagos y rechazos** asumimos?

4.7

Pruebas de hipótesis: H_o, p-valor, errores tipo I y II, potencia

La pregunta

Botón nuevo: conversión **9,91 % → 11,48 %**

¿Mejora real o azar?

Lógica de reducción al absurdo:

1. Supón que **no** hay efecto
2. ¿Qué probabilidad tendrían unos datos así?
3. Si es muy baja, rechaza el supuesto

Hipótesis y estadístico

- H_0: $p_A = p_B$ (no pasa nada)
- H_1: $p_A \neq p_B$ (bilateral) — decidido **antes** de ver datos

$$z = \frac{\hat{p}_B - \hat{p}_A}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_A} + \frac{1}{n_B}\right)}}$$

Notebook: $z = 4{,}39$

El p-valor

> Probabilidad de observar una diferencia **al menos tan grande** como la observada **si H_0 fuera cierta**

- Es una probabilidad sobre los **datos**, suponiendo H_0
- Notebook: $p \approx 1{,}1 \cdot 10^{-5}$
- Si $p < \alpha$ (fijado antes, p. ej. 0,05) ⇒ rechazamos H_0

Lo que el p-valor NO es

✘ Frase incorrecta	Por qué
«97 % de prob. de que B sea mejor» (p = 0,03)	No es $P(H_1 \mid \text{datos})$
«3 % de prob. de que H_0 sea cierta»	No es $P(H_0 \mid \text{datos})$
«p muy pequeño = efecto enorme»	No mide el tamaño
«p = 0,20 demuestra que no hay efecto»	Puede faltar potencia

Errores tipo I y II

	H_0 **cierta**	H_0 **falsa**
Rechazo	Error tipo I (FP) · α	Potencia $1 - \beta$
No rechazo	Acierto	Error tipo II (FN) · β

La misma matriz de confusión... aplicada a experimentos

Potencia: de qué depende

Probabilidad de detectar un efecto **que existe** (objetivo típico: 80 %)

Factor	Si aumenta...
Tamaño del efecto	↑ potencia
Tamaño muestral	↑ potencia
Exigencia (↓ α)	↓ potencia
Variabilidad	↓ potencia

Tamaño muestral (notebook §5)

$$n \approx \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 \left[p_A(1-p_A) + p_B(1-p_B)\right]}{(p_B - p_A)^2}$$

```
efecto = proportion_effectsize(0.11, 0.10)
NormalIndPower().solve_power(effect_size=efecto, alpha=0.05,
                             power=0.8)   # 14.745 por grupo
```

Mitad de efecto ⇒ **4 veces** más muestra

Para llevarse

- p-valor = $P(\text{datos tan extremos} \mid H_0)$
- Significativo ≠ relevante
- No rechazar ≠ demostrar que no hay efecto
- Calcula n **antes** de empezar
- Acompaña siempre el p-valor del **efecto y su IC**

4.8

Pruebas A/B de principio a fin

¿Qué es una prueba A/B?

- Experimento controlado **aleatorizado**
- A = control (actual) · B = tratamiento (nuevo)
- El azar iguala todo lo demás ⇒ diferencia **causal**
- Del «correlación no es causalidad» (módulo 01) a la causalidad medida

Fase 1 · Diseño

1. Hipótesis de negocio
2. Unidad de aleatorización: **usuario**
3. **Una** métrica principal (OEC)
4. Métricas de **salvaguarda**
5. Efecto mínimo detectable (MDE)
6. α, potencia, n y **duración**

Todo por escrito: **plan de análisis**

Tamaño y duración

- Notebook: 10 % → 11 %, α = 0,05, potencia 80 %
- **14.745** usuarios por grupo · 29.490 en total
- Con 3.000 visitas/día → 10 días
- Redondea a **semanas completas** → 2 semanas
- El MDE es el efecto que **vale la pena**, no el que «esperas»

Fase 2 · Ejecución

- Comprueba el reparto: ¿50/50 real?
- **SRM** (*sample ratio mismatch*): prueba chi-cuadrado sobre los recuentos
- Si hay SRM → resultados no fiables
- Vigila errores graves (que B no rompa el pago)
- **No** decidas cuándo parar mirando el p-valor

Fase 3 · Análisis (notebook §6)

```
z, p = proportions_ztest([conv_B.sum(), conv_A.sum()], [n, n])
```

	A	B
Usuarios	15.000	15.000
Conversión	9,91 %	11,48 %

Diferencia **+1,57 p.p.** · mejora relativa **+16 %** · $p \approx 1{,}1 \cdot 10^{-5}$

El IC 95 % de la diferencia

$$(\hat{p}_B - \hat{p}_A) \pm 1{,}96\sqrt{\frac{\hat{p}_A(1-\hat{p}_A)}{n_A} + \frac{\hat{p}_B(1-\hat{p}_B)}{n_B}}$$

[+0,87 ; +2,27] puntos

- Excluye el 0 ⇒ coherente con $p < 0{,}05$
- Escenario pesimista: **sigue siendo positivo**
- Ojo: solapar IC por grupo ≠ no significativo

Fase 4 · Decisión

El IC 95 %...	Lectura	Decisión
Entero por encima del MDE	Real y relevante	Lanzar
Excluye 0, incluye < MDE	Real, tamaño incierto	Lanzar si es barato
Incluye 0 y es estrecho	Sin efecto relevante	No lanzar
Incluye 0 y es ancho	No sabemos	Falta potencia

+ revisar **salvaguardas**

Plantilla de informe

- **Hipótesis:** el botón nuevo aumenta la conversión
- **Diseño:** 50/50 por usuario, 15.000/grupo, MDE +1 p.p., 2 semanas
- **Resultado:** +1,57 p.p. (IC 95 %: +0,87 a +2,27), +16 %, p < 0,001
- **Calidad:** sin SRM, salvaguardas estables
- **Recomendación:** lanzar B · impacto estimado en €

4.9

Trampas: comparaciones múltiples, *peeking*, efecto novedad

Trampa 1 · Comparaciones múltiples

20 pruebas al 5 %, sin ningún efecto real:

$$P(\text{al menos un falso positivo}) = 1 - 0{,}95^{20} \approx 64\,\%$$

- Muchas métricas, muchos segmentos, muchas variantes…
- Probar hasta que algo salga = ***p-hacking***

La simulación del notebook (§7)

1.000 experimentos × 20 métricas sin efecto

Análisis	Experimentos con algún falso positivo
Sin corregir	**65 %**
Corrección de Holm	**4 %**

Cómo defenderse

- **Una** métrica principal, declarada antes
- Corregir si analizas varias:
  - **Bonferroni**: α/m (conservador)
  - **Holm**: escalonado, más potente
  - **Benjamini-Hochberg**: controla la FDR
- Segmento inesperado ⇒ **hipótesis** para otro experimento

```
multipletests(p_valores, alpha=0.05, method="holm")
```

Trampa 2 · *Peeking*

- Mirar el p-valor cada día y parar al bajar de 0,05
- Sin efecto, el p-valor **fluctúa** como un paseo aleatorio
- 20 miradas = 20 oportunidades de «acertar» por azar

Notebook §8: falsos positivos = **27 %** (debería ser 5 %)

Cómo evitar el *peeking*

- Fija n **antes** y analiza **una vez**
- ¿Necesitas parar antes? → métodos **secuenciales**
  - análisis intermedios planificados (O'Brien-Fleming)
  - pruebas secuenciales siempre válidas
- Mirar para detectar errores: sí
- Decidir cuándo parar por el p-valor: **no**

Trampa 3 · Efecto novedad (y primacía)

- **Novedad**: clics por curiosidad que se desvanecen
- **Primacía**: rechazo inicial al cambio que luego se pasa
- Detecta: dibuja el efecto **semana a semana**
- Separa usuarios **nuevos** y **recurrentes**
- Mitiga: más duración, grupo *holdout* a largo plazo

Otras trampas y checklist final

- **SRM**: reparto distinto del diseñado
- **Interferencia** entre grupos (marketplaces)
- **Temporada**: Black Friday ≠ marzo
- **Ley de Twyman**: si parece demasiado bueno, revisa los datos

✔ Plan previo · ✔ una métrica · ✔ n fijado · ✔ efecto + IC · ✔ salvaguardas

Resumen del módulo

Clasificar = estimar probabilidades + decidir con un umbral según costes

Experimentar = planificar n, analizar una vez, reportar efecto + IC + p-valor

Siguiente: caso práctico (umbral + informe A/B) y cuestionario